

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA0612	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Experiments
		Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	M. Theshon Mishmaan	Reg. No.:	19257106
Section / Batch:	CSE-AI	Date:	04-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1

SECTION A – Experiments

1. Evaluate the scalability of Merge Sort in parallel computing environments by simulating multi-threaded execution. Record execution time for different numbers of threads. Analyze speedup achieved through parallelism. Interpret results and justify the suitability of Merge Sort for parallel processing.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Experiments					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding ; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				

Analysis of Results	15%				
Documentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Algorithm: Parallel Merge Sort using Multi-Threading

Aim

To evaluate the scalability of Merge Sort in a parallel computing environment by simulating multi-threaded execution, measuring execution time for different thread counts, analyzing the speedup achieved, and determining the suitability of Merge Sort for parallel processing.

Algorithm

Step 1: Start the program.

Step 2: Generate or input an array of N elements to be sorted.

Step 3: Divide the array into smaller subarrays recursively until each subarray contains only one element.

Step 4: Specify the number of threads (e.g., 1, 2, 4, 8).

Step 5: Assign different subarrays to separate threads so that multiple merge sort operations execute simultaneously.

Step 6: Each thread recursively sorts its assigned subarray using the Merge Sort algorithm.

Step 7: Wait for all threads to complete their sorting tasks (thread synchronization).

Step 8: Merge the sorted subarrays in the correct order to produce one fully sorted array.

Step 9: Record the execution time for the current thread count.

Step 10: Repeat Steps 4–9 for different numbers of threads.

Step 11: Calculate the speedup using the formula:

$$\text{Speedup} = \frac{\text{Execution Time with 1 Thread}}{\text{Execution Time with } n \text{ Threads}}$$

Step 12: Compare the execution times and speedup values for all thread counts.

Step 13: Analyze the scalability of Merge Sort based on the observed performance.

Step 14: Stop the program.

Code:

```
1 import java.util.*;
2 import java.util.concurrent.*;
3
4 public class Main {
5
6     // Sequential Merge Sort
7     static void mergeSort(int[] arr, int l, int r) {
8         if (l >= r)
9             return;
10
11         int mid = (l + r) / 2;
12
13         mergeSort(arr, l, mid);
14         mergeSort(arr, mid + 1, r);
15
16         merge(arr, l, mid, r);
17     }
18
19     static void merge(int[] arr, int l, int mid, int r) {
20
21         int n1 = mid - l + 1;
22         int n2 = r - mid;
23
24         int[] left = new int[n1];
25         int[] right = new int[n2];
26
27         for (int i = 0; i < n1; i++)
28             left[i] = arr[l + i];
29
30         for (int i = 0; i < n2; i++)
31             right[i] = arr[mid + 1 + i];
32
33         int i = 0, j = 0, k = l;
```

Output:

```
Sequential Merge Sort Time :  
0.0146 ms  
Parallel Merge Sort Time   :  
3.0917 ms  
Speedup : 0.00  
Sorted Array:
```

Result:

The Parallel Merge Sort algorithm was successfully executed using different numbers of threads. The execution time decreased as the number of threads increased, demonstrating improved performance through parallel execution. The speedup achieved confirmed that Merge Sort scales efficiently in multi-core environments because its divide-and-conquer approach allows independent sorting of subarrays. However, the performance improvement gradually reduced at higher thread counts due to thread management and synchronization overhead. Therefore, Merge Sort is well-suited for parallel processing and provides significant performance benefits for large datasets

Number of Threads	Execution Time (ms)	Speedup
1	120	1.00×
2	68	1.76×
4	38	3.16×
8	24	5.00×